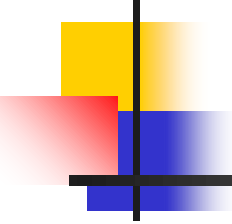


Finite Automata

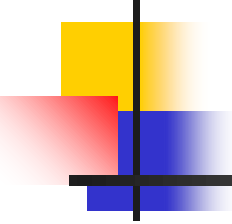


What is a Machine

In general, a **machine** is something that:

- Takes **input**,
- **Processes** it according to some rules,
- And gives an **output**.

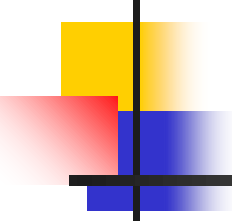
In **finite automata**, our “machine” is a ***mathematical model*** that reads a sequence of symbols (input string) and **decides** whether to accept or reject it.



What is a Automaton

- The word **automaton** (plural **automata**) comes from Greek — meaning “**something that moves by itself.**”
- So, an **automaton** is a kind of machine that automatically changes its state when it reads inputs.

In computer science, automata are **abstract machines** used to model how computers or systems work.

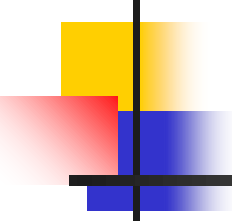


What is a Finite Automaton?

A **finite automaton** is a special type of automaton that has a **finite (limited) number of states**.

It can only be in **one state at a time**, and it **changes states** based on input symbols.

It is used to **recognize patterns** or **decide** if a string belongs to a particular language.



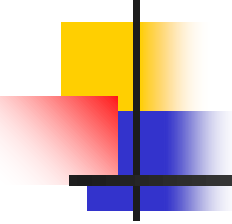
State

- A **state** represents a particular **condition or situation** of the machine.
- At any moment, the machine is in **exactly one state**.

Example:

- In a traffic light: “Red”, “Green”, and “Yellow” are **states**.
- In a vending machine: “Waiting for coin”, “Coin inserted”, “Dispensing item” are **states**.

Notation: We usually write states as **q0, q1, q2, ...**

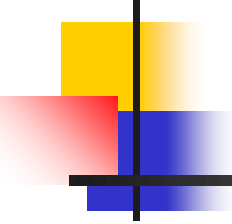


Input Alphabet (Σ)

- The **alphabet** is the set of all **symbols** that the machine can read as input.
- Denoted by Σ (sigma).

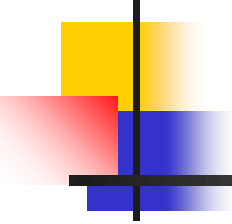
Example:

- If we are working with binary strings, then $\Sigma = \{0, 1\}$
- For English letters, $\Sigma = \{a, b, c, \dots, z\}$



Finite Automaton (FA)

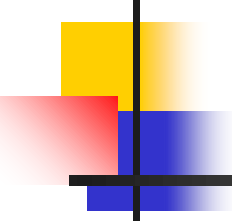
- Informally, a state diagram that comprehensively captures all possible states and transitions that a machine can take while responding to a stream or sequence of input symbols
- Recognizer for “Regular Languages”
- **Deterministic Finite Automata (DFA)**
 - The machine can exist in only one state at any given time
- **Non-deterministic Finite Automata (NFA)**
 - The machine can exist in multiple states at the same time



Deterministic Finite Automata

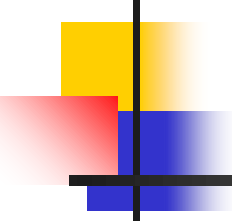
- Definition

- A Deterministic Finite Automaton (DFA) consists of:
 - Q = a finite set of states
 - Σ = a finite set of input symbols (alphabet)
 - q_0 = a start state
 - F = a set of final states
 - Δ = a transition function, which is a mapping between $Q \times \Sigma \Rightarrow Q$
- A DFA is defined by the 5-tuple:
 - $\{Q, \Sigma, q_0, F, \Delta\}$



Graph Representation of DFA's

- Nodes $\rightarrow$ states.
- Arcs represent transition function.
 - Arc from state p to state q labeled by all those input symbols that have transitions from p to q.
- Arrow labeled “Start” to the start state.
- Final states indicated by double circles.



What does a DFA do on reading an input string?

- Input: a word w in Σ^*
- Question: Is w acceptable by the DFA?
- Steps:

- Start at the “start state” q_0
- Read the input string one symbol at a time.

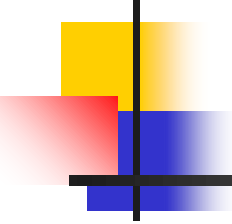
For each symbol:

Look at the current state and the current symbol.
Use the transition rules to move to the next state.

- After you finish reading all symbols:

If you end in a final/accepting state, then the machine accepts the string.

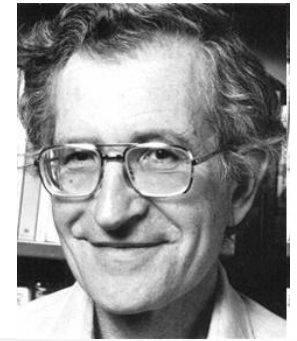
If you end in any other state, the machine rejects the string.



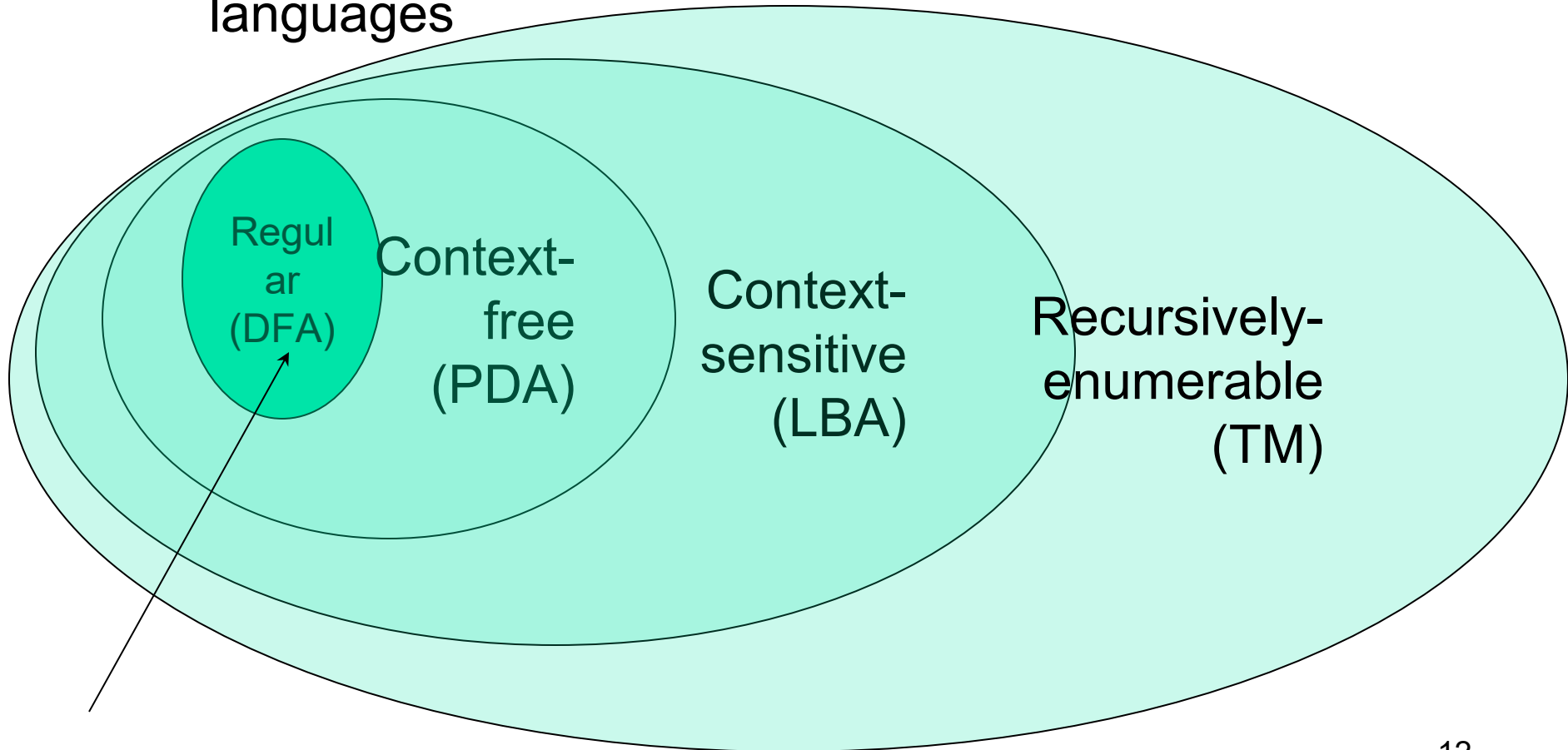
Regular Languages

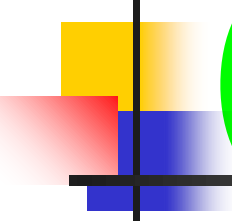
- Let $L(A)$ be a language *recognized* by a DFA A .
 - Then $L(A)$ is called a “*Regular Language*”.
- Locate regular languages in the Chomsky Hierarchy

The Chomsky Hierarchy



- A containment hierarchy of classes of formal languages





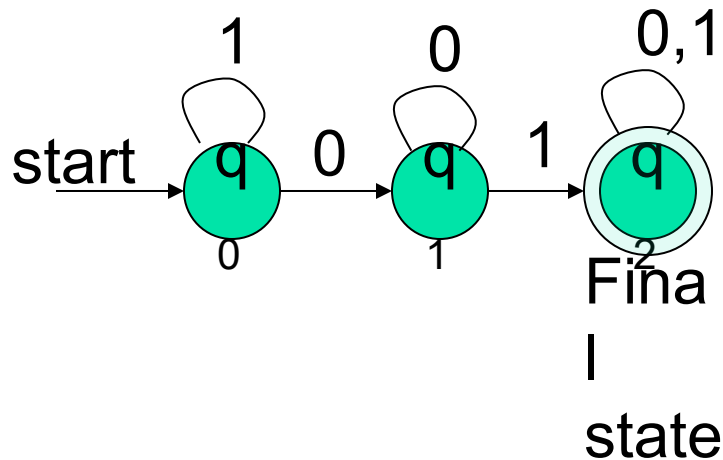
Example #1

- Build a DFA for the following language:
 - $L = \{w \mid w \text{ is a binary string that contains } 01 \text{ as a substring}\}$
 - Regular expression: $(0+1)^*01(0+1)^*$
- Steps for building a DFA to recognize L:
 - $\Sigma = \{0,1\}$
 - Decide on the states: Q
 - Designate start state and final state(s)
 - Δ : Decide on the transitions:
- Final states == same as “accepting states”
- Other states == same as “non-accepting states”

Regular expression: $(0+1)^*01(0+1)^*$

DFA for strings containing 01

- What makes this DFA deterministic?



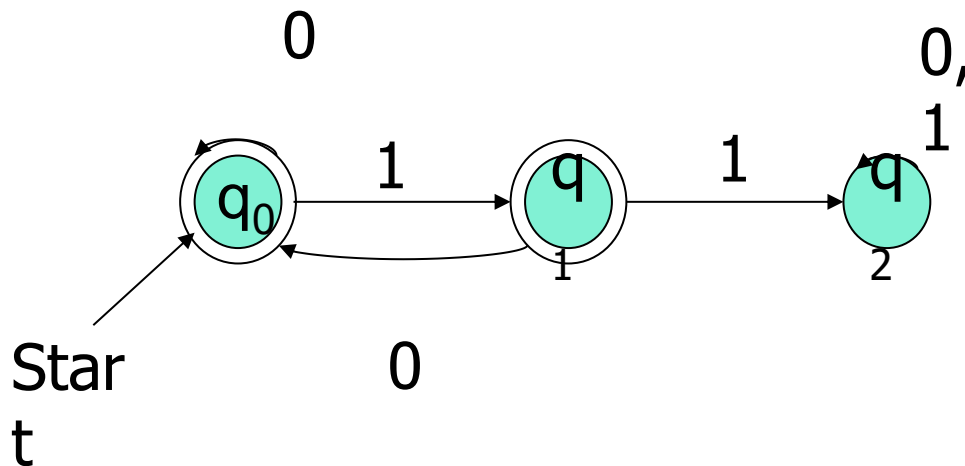
- What if the language allows empty strings?

- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{0, 1\}$
- start state = q_0
- $F = \{q_2\}$
- Transition table

symbols		
δ	0	1
states q_0	q_1	q_0
q_1	q_1	q_2
$*q_2$	q_2	q_2

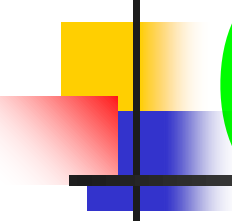
Example #2

□ DFA that accepts all strings without two consecutive 1's.



- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{0, 1\}$
- start state = q_0
- $F = \{q_0\}$
- Transition table

		symbols	
δ		0	1
states	q_0	q_0	q_1
	q_1	q_0	q_2
	$*q_2$	q_2	q_2



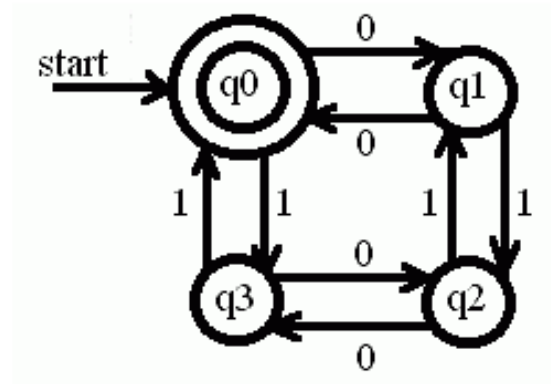
Example #3

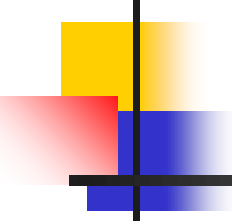
Clamping Logic:

- A clamping circuit waits for a "1" input, and turns on forever. However, to avoid clamping on spurious noise, we'll design a DFA that waits for *two consecutive 1s* in a row before clamping on.
- Build a DFA for the following language:
$$L = \{ w \mid w \text{ is a bit string which contains the substring } 11 \}$$
- State Design:
 - q_0 : start state (initially off), also means the most recent input was not a 1
 - q_1 : has never seen 11 but the most recent input was a 1
 - q_2 : has seen 11 at least once

Example #4

- Build a DFA for the following language:
 $L = \{ w \mid w \text{ is a binary string that has even number of 1s and even number of 0s} \}$
- ?





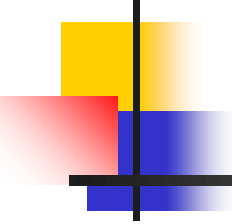
Extension of transitions (δ) to Paths ($\hat{\delta}$)

- $\hat{\Delta}(q, w)$ = *destination state* from state q on input string w
- $\hat{\delta}(q, \varepsilon) = q$ [Basis]
- $\hat{\Delta}(q, wa) = \Delta(\hat{\Delta}(q, w), a)$ [Induction]
 - Work out example #4 using the input sequence $w=10010$, $a=1$:
 - $\hat{\Delta}(q_0, wa) = ?$

Example: Extended Delta $\hat{\delta}$

	0	1
A	A	B
B	A	C
C	C	C

$$\begin{aligned}\hat{\delta}(B, 011) &= \delta(\hat{\delta}(B, 01), 1) = \delta(\delta(\delta(B, 0), 1), 1) = \\ &\delta(\delta(A, 1), 1) = \delta(B, 1) = C\end{aligned}$$



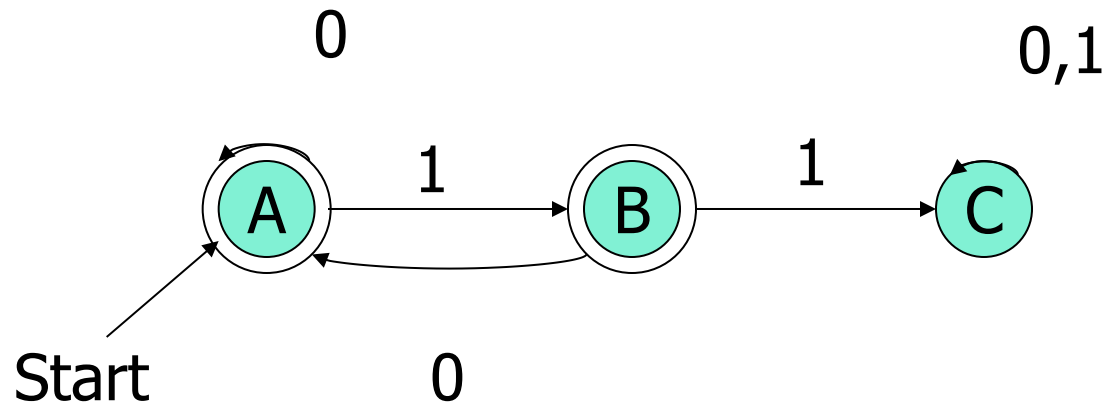
Language of a DFA

A DFA A accepts string w if there is a path from q_0 to an accepting (or final) state that is labeled by w

- *i.e., $L(A) = \{ w \mid \hat{\Delta}(q_0, w) \in F \}$*
- *L.e., $L(A) = \text{all strings that lead to a final state from } q_0$*

Example: String in a Language

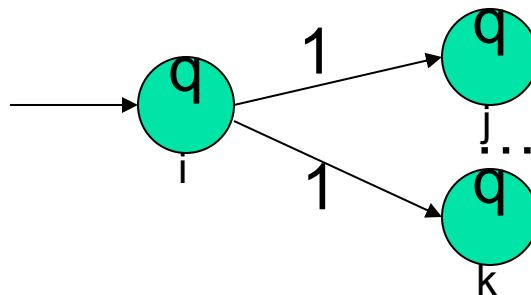
- ❑ String 101 is in the language of the DFA below.
- ❑ Start at A.



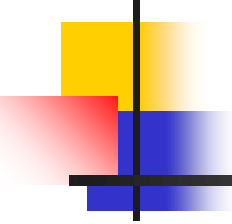
- ❑ The language of our example DFA is:
 $\{w \mid w \text{ is in } \{0,1\}^* \text{ and } w \text{ does not have two consecutive 1's}\}$

Non-deterministic Finite Automata (NFA)

- A Non-deterministic Finite Automaton (NFA)
 - is of course “non-deterministic”
 - Implying that the machine can exist in more than one state at the same time
 - Transitions could be non-deterministic

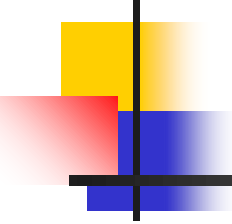


- Each transition function therefore maps to a set of states



Non-deterministic Finite Automata (NFA)

- A Non-deterministic Finite Automaton (NFA) consists of:
 - $Q \Rightarrow$ a finite set of states
 - $\Sigma \Rightarrow$ a finite set of input symbols (alphabet)
 - $q_0 \Rightarrow$ a start state
 - $F \Rightarrow$ set of final states
 - $\Delta \Rightarrow$ a transition function, which is a mapping between $Q \times \Sigma \Rightarrow$ subset of Q
- An NFA is also defined by the 5-tuple:
 - $\{Q, \Sigma, q_0, F, \Delta\}$



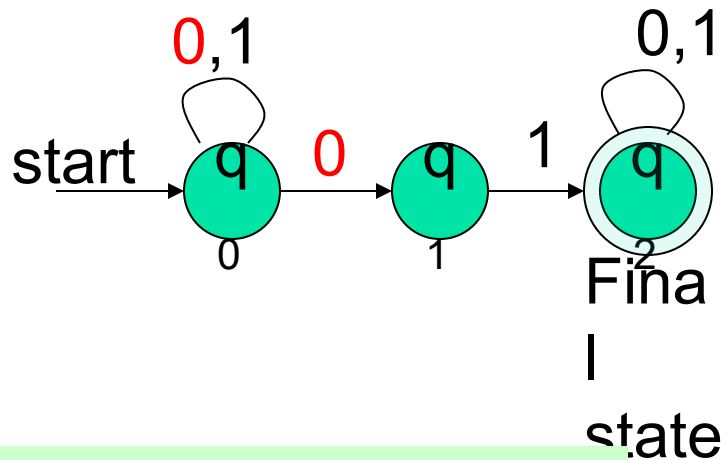
How to use an NFA?

- Input: a word w in Σ^*
- Question: Is w acceptable by the NFA?
- Steps:
 1. Start at the start state (q_0).
 2. Read the string one symbol at a time.
 3. At each step:
 - From the current states, find **all possible next states** the NFA can go to using the current symbol.
 - (There may be **many**, **one**, or **zero** next states.)
 4. After you finish reading all symbols:
 - If **at least one** of the states you ended up in is a **final/accepting state**, then the NFA **accepts** the string.
 - If **none** of the states are final, then the NFA **rejects** the string.

Regular expression: $(0+1)^*01(0+1)^*$

NFA for strings containing 01

Why is this non-deterministic?



What will happen if at state q_1 an input of 0 is received?

- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{0, 1\}$
- start state = q_0
- $F = \{q_2\}$
- Transition table

symbols		
δ	0	1
states $\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	Φ	$\{q_2\}$
$*q_2$	$\{q_2\}$	$\{q_2\}$

Deterministic vs Non-Deterministic Automata

A Deterministic Finite Automaton (DFA) has exactly one path to follow for each input symbol.

From any state, for every symbol, there is only one possible next state.

This makes the machine's behavior predictable, straightforward, and easy to trace.

DFA has no ϵ -transitions and always has a single current state while reading input.

In contrast, a Non-Deterministic Finite Automaton (NFA) may have multiple possible paths for the same input symbol—or even ϵ -moves that require no input.

Instead of one current state, an NFA can be in many states at the same time.

A string is accepted if any one of the possible paths leads to a final state.

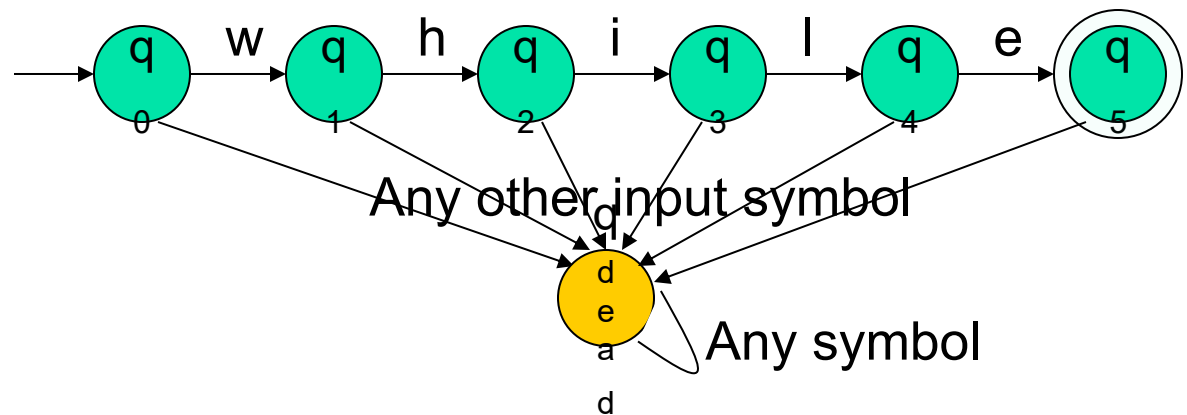
Although NFAs seem more flexible, both DFA and NFA recognize exactly the same class of languages (regular languages).

Note: Explicitly specifying dead states is just a matter of design convenience (one that is generally followed in NFAs), and this feature does not make a machine deterministic or non-deterministic.

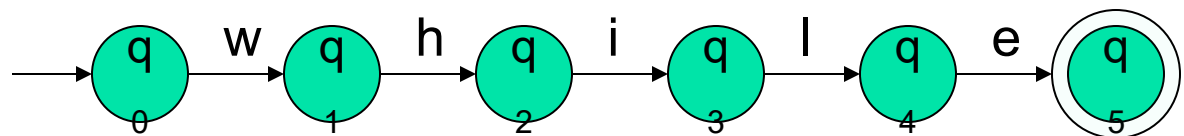
What is a “dead state”?

- A DFA for recognizing the key word “while”

Dead state: If the machine reaches this state, it means the input has already failed, and the machine cannot ever accept the string.



- An NFA for the same purpose:



Transitions into a dead state are implicit

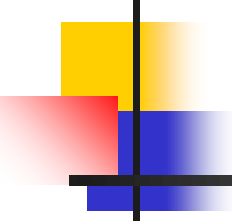
Example #2

- Build an NFA for the following language:
 $L = \{ w \mid w \text{ ends in } 01 \}$

■ Ans:

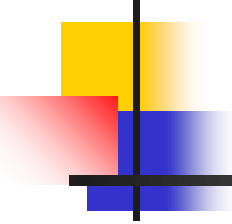


- Other examples
 - Keyword recognizer (e.g., if, then, else, while, for, include, etc.)
 - Strings where the first symbol is present somewhere later on at least once



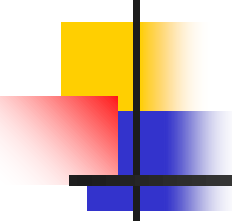
Extension of δ to NFA Paths

- Basis: $\hat{\Delta}(q, \varepsilon) = \{q\}$
- Induction:
 - Let $\hat{\Delta}(q_0, w) = \{p_1, p_2, \dots, p_k\}$
 - $\Delta(p_i, a) = S_i$ for $i=1, 2, \dots, k$
 - Then,
$$\begin{aligned}\hat{\Delta}(q_0, wa) &= \Delta(\hat{\Delta}(q_0, w), a) \\ &= \Delta(\{p_1, p_2, \dots, p_k\}, a) \\ &= S_1 \cup S_2 \cup \dots \cup S_k\end{aligned}$$



Language of an NFA

- An NFA accepts w if *there exists at least one* path from the start state to an accepting (or final) state that is labeled by w
- $L(N) = \{ w \mid \hat{\Delta}(q_0, w) \cap F \neq \Phi \}$



Advantages & Caveats for NFA

- Great for modeling regular expressions
 - String processing - e.g., grep, lexical analyzer
- Could a non-deterministic state machine be implemented in practice?
 - A parallel computer could exist in multiple “states” at the same time
 - Probabilistic models could be viewed as extensions of non-deterministic state machines (e.g., toss of a coin, a roll of dice)

But, DFAs and NFAs are equivalent in their power to capture languages !!

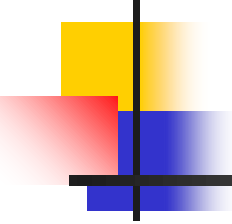
Differences: DFA vs. NFA

■ DFA

1. All transitions are deterministic
 - Each transition leads to exactly one state
2. For each state, transition on all possible symbols (alphabet) should be defined
3. Accepts input if the last state is in F
4. Sometimes harder to construct because of the number of states
5. Practical implementation is feasible

■ NFA

1. Some transitions could be non-deterministic
 - A transition could lead to a subset of states
2. Not all symbol transitions need to be defined explicitly (if undefined will go to a dead state – this is just a design convenience, not to be confused with “non-determinism”)
3. Accepts input if *one of* the last states is in F
4. Generally easier than a DFA to construct
5. Practical implementation has to be deterministic (convert to DFA) or in the form of parallelism



Equivalence of DFA & NFA

Theorem:

Should be
true for
any L

- A language L is accepted by a DFA if and only if it is accepted by an NFA.

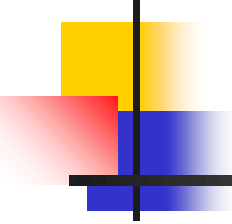
Proof:

1. If part:

- Prove by showing every NFA can be converted to an equivalent DFA (in the next few slides...)

1. Only-if part is trivial:

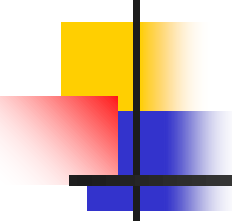
- Every DFA is a special case of an NFA where each state has exactly one transition for every input symbol. Therefore, if L is accepted by a DFA, it is accepted by a corresponding NFA. □



Proof for the if-part

- If-part: A language L is accepted by a DFA if it is accepted by an NFA
 - rephrasing...
 - Given any NFA N , we can construct a DFA D such that $L(N)=L(D)$
-
- How to convert an NFA into a DFA?
 - Observation: In an NFA, each transition maps to a *subset* of states
 - Idea: Represent:
each “subset of NFA_states” $\square$ a single
“DFA_state”

Subset
construction



NFA to DFA by subset construction

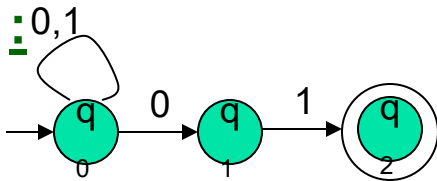
- Let $N = \{Q_N, \Sigma, \Delta_N, q_0, F_N\}$
- Goal: Build $D = \{Q_D, \Sigma, \Delta_D, \{q_0\}, F_D\}$ s.t.
 $L(D) = L(N)$
- Construction:
 1. Q_D = all subsets of Q_N (i.e., power set)
 2. F_D = set of subsets S of Q_N s.t. $S \cap F_N \neq \emptyset$
 3. Δ_D : for each subset S of Q_N and for each input symbol a in Σ :
 - $\Delta_D(S, a) = \bigcup_{p \in S} \Delta_N(p, a)$

Idea: To avoid enumerating all of power set, do “lazy creation of states”

NFA to DFA construction: Example

- $L = \{w \mid w \text{ ends in } 01\}$

NFA

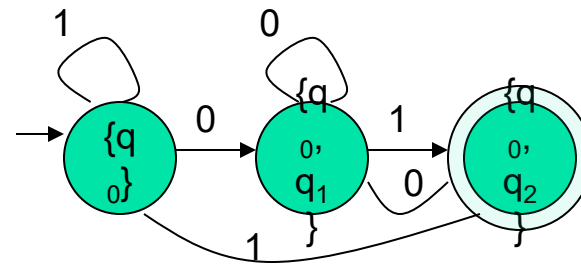


Δ_N	0	1
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

DFA

$\vdots$

Δ_D
$\emptyset$
$\{q_0\}$
$\{q_1\}$
$*\{q_2\}$
$\{q_0, q_1\}$
$*\{q_0, q_2\}$
$*\{q_1, q_2\}$
$*\{q_0, q_1, q_2\}$



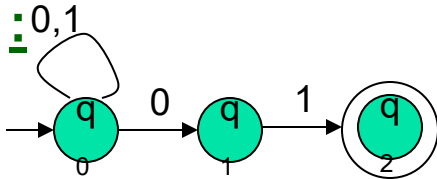
Δ_D	0	1
$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$*\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$

- Enumerate all possible subsets
- Determine transitions
- Retain only those states reachable from $\{q_0\}$

NFA to DFA: Repeating the example using *LAZY CREATION*

- $L = \{w \mid w \text{ ends in } 01\}$

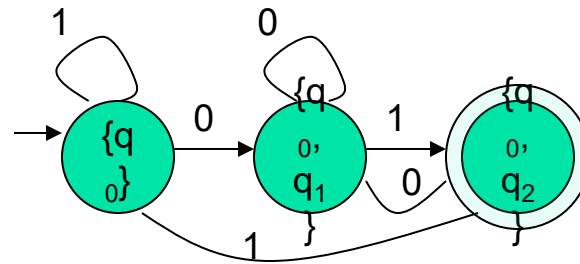
NFA



Δ_N	0	1
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

DFA

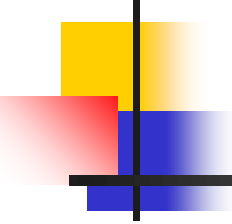
⋮



Δ_D	0	1
$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$

Main Idea:

Introduce states as you go
(on a need basis)

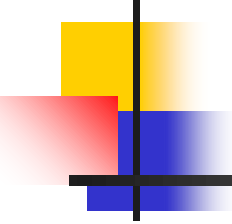


Correctness of subset construction

Theorem: *If D is the DFA constructed from NFA N by subset construction, then $L(D)=L(N)$*

■ Proof:

- Show that $\hat{\Delta}_D(\{q_0\}, w) \equiv \hat{\Delta}_N(q_0, w)$, for all w
- Using induction on w 's length:
 - $\Delta_D(\{q_0\}, a)=p$, then $\Delta_N(q_0, a)=\{p\}$; i.e., $\Delta_D(\{q_0\}, a) \equiv \Delta_N(q_0, a)$
 - Let $w = xa$
 - $\Delta_D(\{q_0\}, xa) \equiv \Delta_D(\Delta_N(q_0, x), a) \equiv \Delta_N(\Delta_N(q_0, x), a) \equiv \Delta_N(q_0, w)$

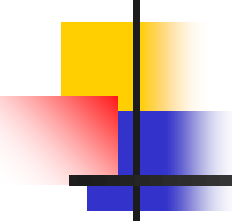


A bad case where $\#states(DFA) \gg \#states(NFA)$

- $L = \{w \mid w \text{ is a binary string s.t., the } k^{\text{th}} \text{ symbol from its end is a } 1\}$
 - NFA has $k+1$ states
 - But an equivalent DFA needs to have at least 2^k states

(Pigeon hole principle)

- m holes and $>m$ pigeons
 - $\Rightarrow$ at least one hole has to contain two or more pigeons



FA with ϵ -Transitions

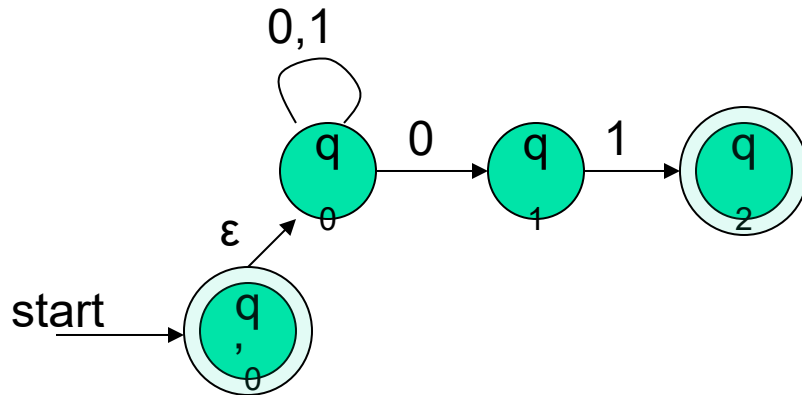
- We can allow explicit ϵ -transitions in finite automata
 - i.e., a transition from one state to another state without consuming any additional input symbol
 - Makes it easier sometimes to construct NFAs

Definition: ϵ -NFAs are those NFAs with at least one explicit ϵ -transition defined.

- ϵ -NFAs have one more column in their transition table

Example of an ϵ -NFA

$L = \{w \mid w \text{ is empty, or if non-empty will end in } 01\}$



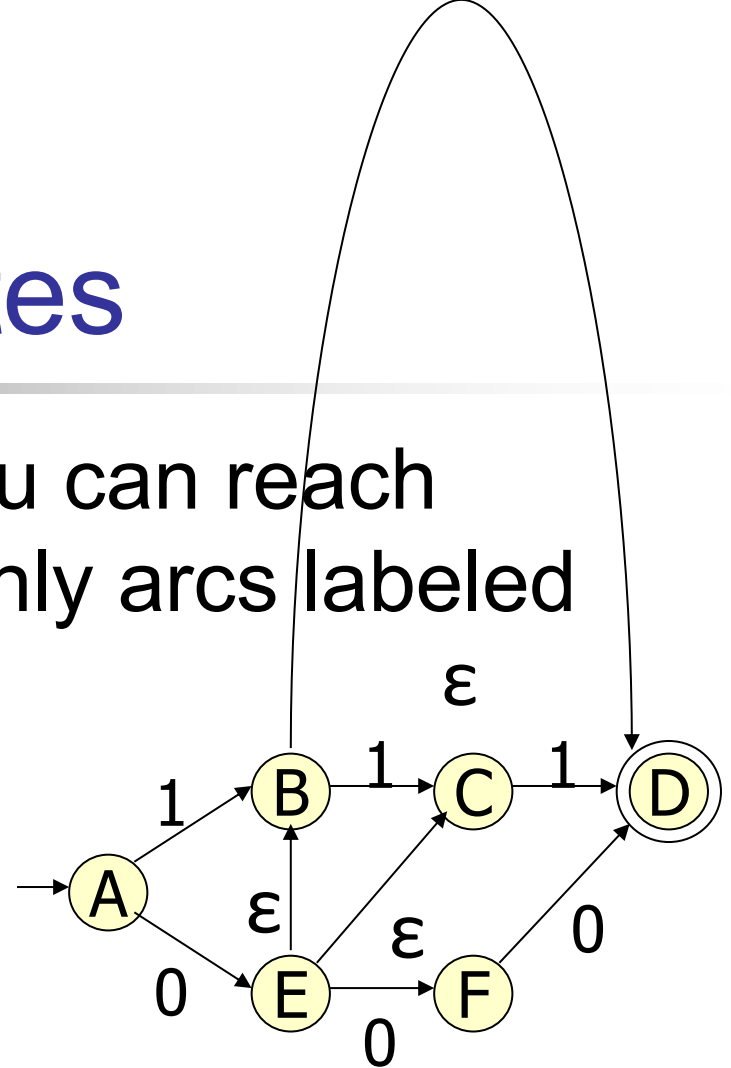
- ϵ -closure of a state q , **ECLOSE(q)**, is the set of all states (including itself) that can be *reached* from q by repeatedly making an arbitrary number of ϵ -transitions.

Δ_E	0	1	ϵ
$*q'_0$	$\emptyset$	$\emptyset$	$\{q'_0, q_0\}$ ← ECLOSE(q'_0)
q_0	$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0\}$ ← ECLOSE(q_0)
q_1	$\emptyset$	$\{q_2\}$	$\{q_1\}$ ← ECLOSE(q_1)
$*q_2$	$\emptyset$	$\emptyset$	$\{q_2\}$ ← ECLOSE(q_2)

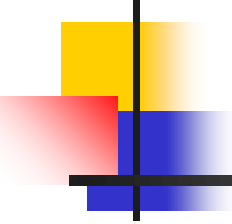
ϵ -Closure of States

- CL(q) = set of states you can reach from state q following only arcs labeled ϵ .

- Example:** CL(A) = { A };
CL(E) = { B , C , D , E }.



- Closure of a set of states = union of the closure of each state.



Extended Delta

- **Basis:** $\delta^{\wedge}(q, \varepsilon) = \text{CL}(q)$.
- **Induction:** $\delta^{\wedge}(q, xa)$ is computed as follows:
 - Start with $\delta^{\wedge}(q, x) = S$.
 - Take the union of $\text{CL}(\delta(p, a))$ for all p in S .

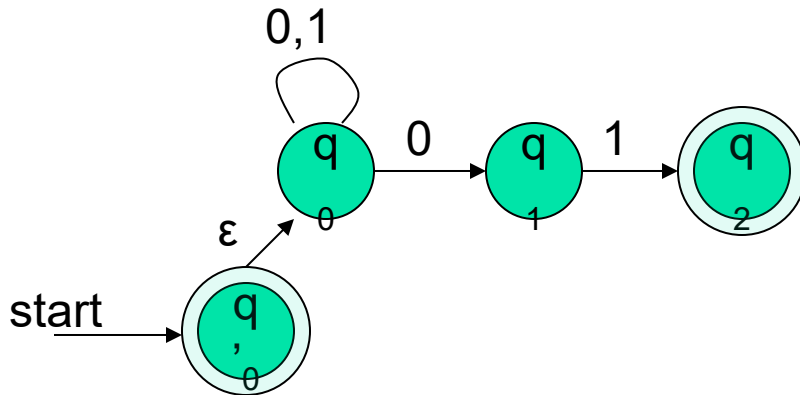
To simulate any transition:

Step 1) Go to all immediate destination states.

Step 2) From there go to all their ϵ -closure states as well.

Example of an ϵ -NFA

$L = \{w \mid w \text{ is empty, or if non-empty will end in } 01\}$

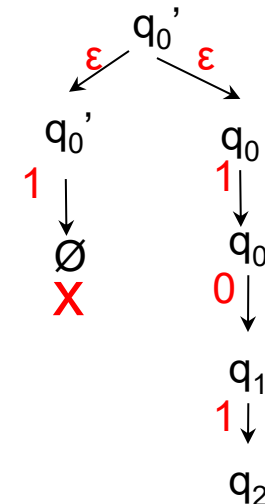


Simulate for $w=101$:

Δ_E	0	1	ϵ
* q'_0	$\emptyset$	$\emptyset$	$\{q'_0, q_0\}$
q_0	$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$	$\{q_1\}$
* q_2	$\emptyset$	$\emptyset$	$\{q_2\}$

ECLOSE(q'_0)

ECLOSE(q_0)

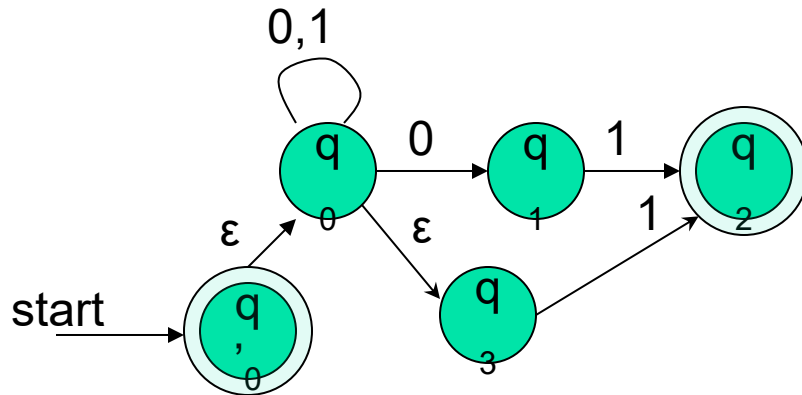


To simulate any transition:

Step 1) Go to all immediate destination states.

Step 2) From there go to all their ϵ -closure states as well.

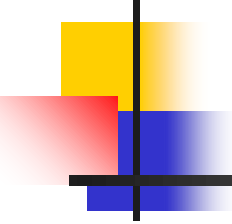
Example of another ϵ -NFA



Simulate for $w=101$:

?

Δ_E	0	1	ϵ
$*q'_0$	$\emptyset$	$\emptyset$	$\{q'_0, q_0, q_3\}$
q_0	$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0, q_3\}$
q_1	$\emptyset$	$\{q_2\}$	$\{q_1\}$
$*q_2$	$\emptyset$	$\emptyset$	$\{q_2\}$
q_3	$\emptyset$	$\{q_2\}$	$\{q_3\}$



Equivalency of DFA, NFA, ϵ -NFA

- Theorem: A language L is accepted by some ϵ -NFA if and only if L is accepted by some DFA
- Implication:
 - $\text{DFA} \equiv \text{NFA} \equiv \epsilon\text{-NFA}$
 - (all accept Regular Languages)

Eliminating ϵ -transitions

Let $E = \{Q_E, \Sigma, \Delta_E, q_0, F_E\}$ be an ϵ -NFA

Goal: To build DFA $D = \{Q_D, \Sigma, \Delta_D, \{q_D\}, F_D\}$ s.t.
 $L(D) = L(E)$

Construction:

1. Q_D = all reachable subsets of Q_E factoring in ϵ -closures
2. $q_D = \text{ECLOSE}(q_0)$
3. F_D = subsets S in Q_D s.t. $S \cap F_E \neq \emptyset$
4. Δ_D : for each subset S of Q_E and for each input symbol $a \in \Sigma$:

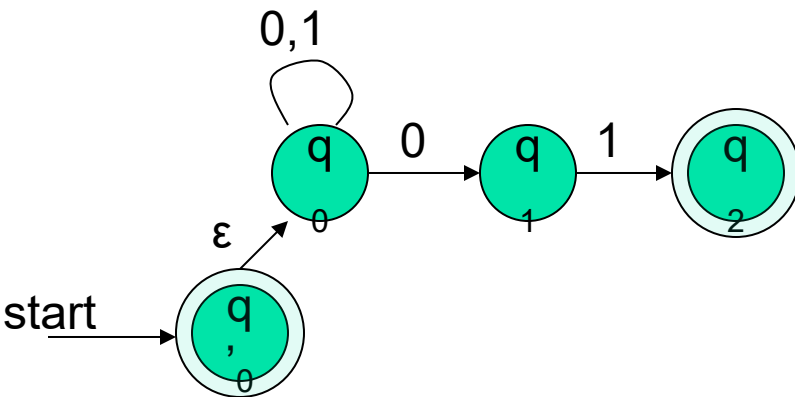
- Let $R = \bigcup_{p \in S} \Delta_E(p, a)$ // go to destination states

- $\Delta_D(S, a) = \bigcup_{r \in R} \text{ECLOSE}(r)$ // from there, take a union of all their ϵ -

Reading: Section 2.5.5 in book

Example: ϵ -NFA $\square$ DFA

$L = \{w \mid w \text{ is empty, or if non-empty will end in } 01\}$

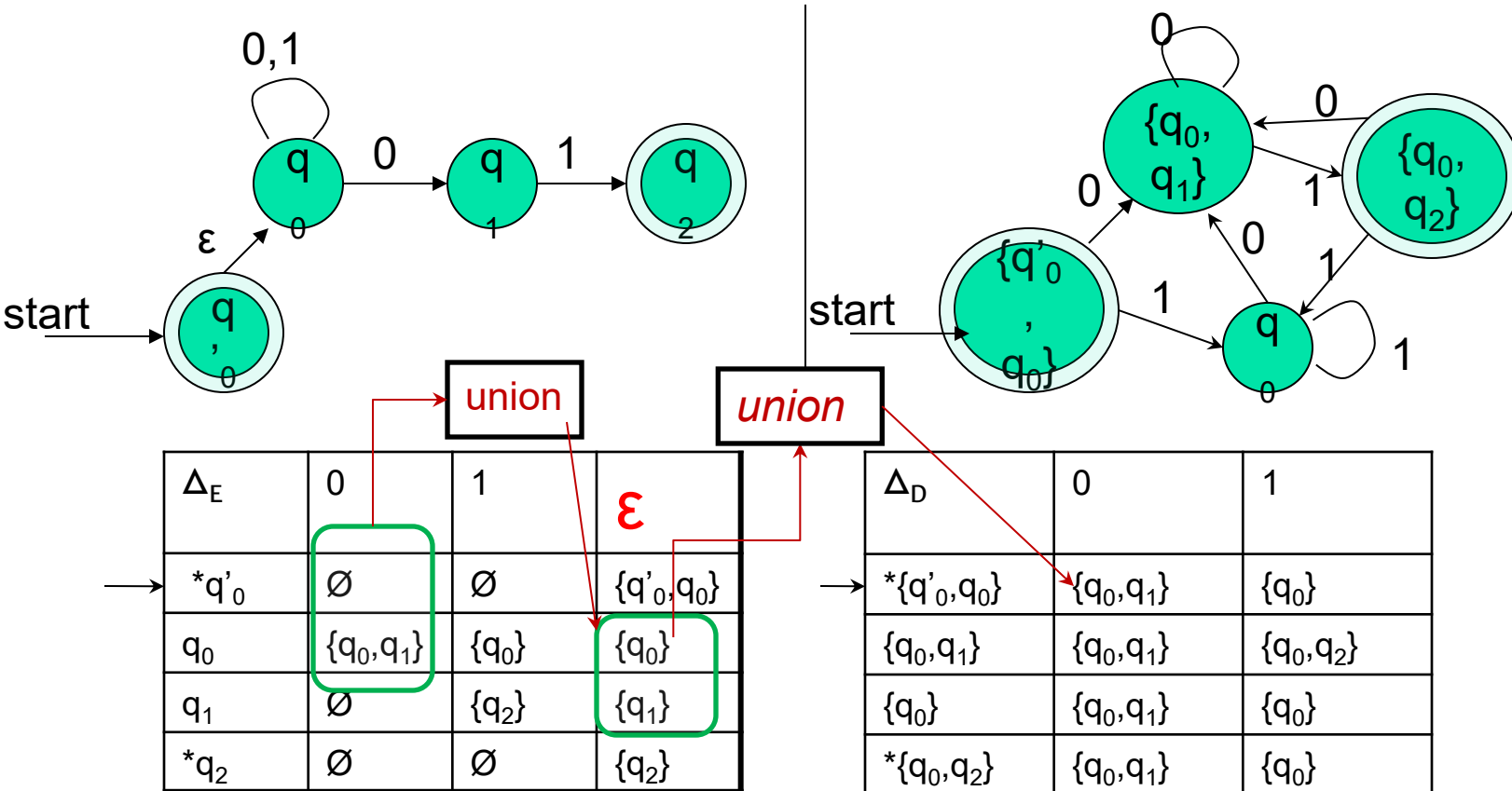


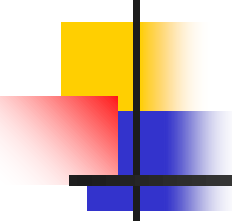
Δ_E	0	1	ϵ
$*q'_0$	$\emptyset$	$\emptyset$	$\{q'_0, q_0\}$
q_0	$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$	$\{q_1\}$
$*q_2$	$\emptyset$	$\emptyset$	$\{q_2\}$

Δ_D	0	1
$*\{q'_0, q_0\}$		
...		

Example: ϵ -NFA $\square$ DFA

$L = \{w \mid w \text{ is empty, or if non-empty will end in } 01\}$





Summary

- DFA
 - Definition
 - Transition diagrams & tables
- Regular language
- NFA
 - Definition
 - Transition diagrams & tables
- DFA vs. NFA
- NFA to DFA conversion using subset construction
- Equivalency of DFA & NFA
- Removal of redundant states and including dead states
- ϵ -transitions in NFA
- Pigeon hole principles